

Disclaimer and Important Notes for the SQL Boot camp : Learn fast , query like a pro Course

This educational course is designed to provide foundational and practical knowledge of SQL. Please read the following carefully:

1. For Educational Purposes Only:

All information, scripts, commands, and materials provided in this course, including all shared scripts, are intended for educational purposes only. Using these scripts or SQL statements in any operational or sensitive environment (including production, testing, or development environments) is strongly discouraged. DO NOT TRY THEM on sensitive environments or databases.

2. No Guarantee of Functionality or Safety:

The scripts and commands provided in this course for educational purposes , may produce unexpected results in various environments. No guarantee is provided regarding the correct functioning, safety, or suitability of these scripts for use in real-world environments.

3. Responsibility for Use:

Any use of the scripts (SQL statements), commands, or concepts presented in this course is the sole responsibility of the participant. The course organizers are not responsible for any damages, disruptions, or issues arising from executing the scripts in any operational or sensitive environment. Participants must fully accept all consequences of using the scripts (SQL statements) and commands.

4. Prohibited Use in Operational and Sensitive Environments:

Executing the scripts (SQL statements), commands, or materials from this course, including the Samples in the final section of this course , in any operational or sensitive environment (including production, testing, or development environments) is strictly prohibited. Participants must first test the scripts in non-operational (educational) environments and accept full responsibility for any incorrect or unauthorized execution.

5. Adherence to Organizational Policies and Standards:

Participants are required to comply with their organization's policies and standards, when using the materials from this course. Using these materials in operational environments should be done with consultation and coordination with technical teams and experts.

6. Intellectual Property Rights:

All materials, scripts (SQL statements), and content provided in this course are the property of the course organizers. Redistribution, publication, or unauthorized use of this content without prior permission is prohibited and subject to legal action.

7. No Guarantee of Learning or Success:

This course is designed to provide practical knowledge; however, no guarantee is made regarding the complete understanding of concepts or the achievement of specific professional skills. Success in learning depends on the participant's effort, practice, and individual background.

Additional Explanation:

This course is designed to teach you SQL (Structured query language). Using the scripts or concepts provided in operational environments requires advanced knowledge, careful review, and consultation with professional experts. Participants must take full responsibility for their use of this Content.

Practice 1: Find Young Female Runners ♦

Task:

Show first_name, last_name, and approximate age of all female runners born after 1996-01-01. Sort by age ascending.

✅ SQL:

```
SELECT first_name , last_name,  
TO_NUMBER(TO_CHAR(CURRENT_DATE, 'YYYY'), '9999') -  
TO_NUMBER(TO_CHAR(birth_date, 'YYYY'), '9999') AS age  
FROM Runners  
WHERE gender = 'F'  
AND birth_date > '1996-01-01'  
ORDER BY age ASC;
```

♦ Practice 2: High-Paying Trail Races

Task:

List all races where the surface_type is 'Trail' and the prize_money is more than 100000. Show race_name, location, and prize_money.

✅ SQL:

```
SELECT race_name,location,prize_money  
FROM Races  
WHERE surface_type = 'Trail'  
AND prize_money > 100000  
ORDER BY prize_money DESC;
```

◆ Practice 3: Fast Finish Times

Task:

Find all finish times faster than 00:20:00. Show race_id, runner_id, and finish_time. Order by fastest time first.

✓ SQL:

```
SELECT race_id , runner_id , finish_time
FROM Race_Results
WHERE finish_time < '00:20:00'
ORDER BY finish_time ASC;
```

◆ Practice 4: Approximate Runner Age Group

Task:

Add a column that groups runners into:

"25 or Under 25" if age =< 25

"26 to 30" if age is between 26 and 30

"Above 30" if older

✓ SQL:

```
SELECT first_name , last_name,
CASE
WHEN TO_NUMBER(TO_CHAR(CURRENT_DATE, 'YYYY'), '9999') -
TO_NUMBER(TO_CHAR(birth_date, 'YYYY'), '9999') <= 25 THEN '25 or Under 25'
WHEN TO_NUMBER(TO_CHAR(CURRENT_DATE, 'YYYY'), '9999') -
TO_NUMBER(TO_CHAR(birth_date, 'YYYY'), '9999') BETWEEN 26 AND 30 THEN '26 to 30'
ELSE 'Above 30'
END AS age_group
FROM Runners;
```

◆ Practice 5 : Identify Fast or Top-Finishing Runners

Scenario:

The race organizers want to analyze high-performing results.

You've been asked to list all the results where a runner either:

Finished in first place,

OR

Finished the race in less than 40 minutes,

BUT the race must not be a long-distance race (i.e., not race_id = 3 or race_id = 5).

Task:

Write a query to return the race_id, runner_id, finish_time, and position from the Race_Results table based on the logic described above.

✓ SQL:

```
SELECT race_id, runner_id, finish_time, position
FROM Race_Results
WHERE
(position = 1 OR finish_time < '00:40:00')
AND race_id NOT IN (3, 5);
```

◆ Practice 6 (STRING FUNCTIONS): Format Runner Names — First Letter Uppercase for Last Name

Scenario:

You want to format runner names so that:

The first letter of the last name is uppercase

The rest of the last name is lowercase

The first name is fully in lowercase

All names are trimmed for extra spaces

Format: Last_Name Formatted, first_name

Task:

Write a query to return:

runner_id and a column formatted_name with the format described

✅ SQL:

```
SELECT
runner_id,
UPPER(SUBSTRING(TRIM(last_name), 1, 1)) || LOWER(SUBSTRING(TRIM(last_name), 2,
LENGTH(TRIM(last_name)) - 1)) || ', ' || LOWER(TRIM(first_name)) AS formatted_name
FROM Runners;
```

◆ Practice 7 :

List All Runners Who Have Never Taken 1st, 2nd, or 3rd Position (Using JOIN)

Task: Show runner_id, first_name, and last_name of all runners who have never finished in positions 1, 2 any race.

✅ SQL:

```
SELECT
r.runner_id,
r.first_name,
r.last_name
FROM runners r
LEFT JOIN race_results rr
ON r.runner_id = rr.runner_id
WHERE rr.position not IN (1, 2)
ORDER BY r.last_name, r.first_name;
```

◆ Practice 8: Show Runners with Fast Finish Times

Task:

Display the first_name, last_name, and finish_time of runners who finished a race in under 25minutes.

✅ SQL:

```
SELECT
r.first_name,
r.last_name,
rr.finish_time
FROM runners r
JOIN race_results rr
ON r.runner_id = rr.runner_id
WHERE rr.finish_time < INTERVAL '25 minutes';
```

◆ Practice 9: Find Runners Who Participated in Trail Races with Prize Money Over 50,000

Task: List first_name, last_name, race_name, and prize_money for runners who participated in races with surface_type = 'Trail' and prize_money > 50000.

✅ SQL:

```
SELECT DISTINCT r.first_name, r.last_name, rce.race_name, rce.prize_money
FROM runners r
JOIN race_results rr ON r.runner_id = rr.runner_id
JOIN races rce ON rr.race_id = rce.race_id
WHERE rce.surface_type = 'Trail'
AND rce.prize_money > 50000;
```

- ◆ Practice 10 : List each race_name with the total number of runners who participated in it.

✓ SQL:

```
SELECT r.race_name , COUNT(rr.runner_id) AS total_runners
FROM races r
JOIN race_results rr ON r.race_id = rr.race_id
GROUP BY r.race_name;
```

- ◆ Practice 11 : Display each race along with its average finish time, ordered from fastest to slowest.

✓ SQL:

```
SELECT r.race_name, AVG(rr.finish_time) AS avg_finish_time
FROM races r
JOIN race_results rr ON r.race_id = rr.race_id
GROUP BY r.race_name
ORDER BY avg_finish_time ASC;
```

- ◆ Practice 12 : Find races that had more than 2 participants.

✓ SQL:

```
SELECT r.race_name, COUNT(rr.runner_id) AS participant_count
FROM races r
JOIN race_results rr ON r.race_id = rr.race_id
GROUP BY r.race_name HAVING COUNT(rr.runner_id) > 2;
```

◆ Practice 13:

For each runner, display their best (minimum) finish time.

✓ SQL:

```
SELECT ru.first_name, ru.last_name, min(rr.finish_time) AS best_finish_time
FROM race_results rr JOIN runners ru ON rr.runner_id = ru.runner_id
GROUP BY ru.first_name, ru.last_name;
```

◆ Practice 14: How many times did each runner finish in first position?

✓ SQL:

```
SELECT r.first_name , r.last_name , COUNT(rr.race_id) AS first_position_count
FROM runners r
JOIN race_results rr ON r.runner_id = rr.runner_id
WHERE rr.position = 1
GROUP BY r.first_name, r.last_name;
```

◆ Practice 15: List each race with the fastest finish time recorded in that race.

✓ SQL:

```
SELECT r.race_name, MIN(rr.finish_time) AS fastest_finish_time
FROM races r JOIN race_results rr ON r.race_id = rr.race_id
GROUP BY r.race_name;
```